The Environment Model

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss the difference between **dynamic** and **lexical** scoping
- Introduce **closures** as a way of implementing lexical scoping in the environment model
- Discuss **recursion** and closures

Recap

Recall: Lambda Calculus

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
```

syntax

Recall: Lambda Calculus

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
```

syntax

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$\frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

small-step call-by-value

Recall: Lambda Calculus

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
```

syntax

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$\frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

small-step call-by-value

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

small-step call-by-name

Recall: Lambda Calculus

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
```

syntax

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$\frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

small-step call-by-value

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

small-step call-by-name

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

big-step call-by-value

Recall: Lambda Calculus

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
```

syntax

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$\frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

small-step call-by-value

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

small-step call-by-name

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

big-step call-by-value

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad [e_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

big-step call-by-name

Recall: Substitution

$$[y/x](\lambda x\,.\,y)$$

Recall: Substitution

$$[y/x](\lambda x\,.\,y)$$

We write $[v/x]e$ to mean e with v substituted in for x

Recall: Substitution

$$[y/x](\lambda x\,.\,y)$$

We write $[v/x]e$ to mean e with v substituted in for x

Informally. *Replace every instance of* x *with* v

Recall: α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x \,.\, \lambda y \,.\, x =_\alpha \lambda v \,.\, \lambda w \,.\, v$$

λ-calculus

Recall: α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x . \lambda y . x =_\alpha \lambda v . \lambda w . v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

Recall: α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x . \lambda y . x =_\alpha \lambda v . \lambda w . v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

We say that a variable x is **bound** is an expression if it appears in the expression as $(\ldots \lambda x . e \ldots)$

Recall: α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x . \lambda y . x =_\alpha \lambda v . \lambda w . v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

We say that a variable x is **bound** is an expression if it appears in the expression as $(\ldots \lambda x . e \ldots)$

Substitution should preserve this

Recall: "Correct" Definition

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Recall: "Correct" Definition

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

The only problem with this definition is that it now poses an *implementation issue*. **How do we come up with z?**

Well-Scopedness and Closedness

open

$$\lambda x . y$$

closed

$$\lambda x . \lambda y . y$$

Def. *(informal)* An expression e is **well-scoped** if every free variable in e is "in scope" (more on that on Thursday)

Def. An expression e is **closed** if it has no free variables

Every closed term is well-scoped

One Solution: Well-Scopedness Check

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

One Solution: Well-Scopedness Check

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

If we only work with closed (well-scoped) expressions, then we don't need to worry about captured variables

One Solution: Well-Scopedness Check

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

If we only work with closed (well-scoped) expressions, then we don't need to worry about captured variables

The condition requiring α-renaming never holds!

One Solution: Well-Scopedness Check

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

If we only work with closed (well-scoped) expressions, then we don't need to worry about captured variables

The condition requiring α-renaming never holds!

The takeaway: during the workshop, we will check that the given expression is *closed*

Variable Scoping

Two Major Concerns

Two Major Concerns

1. Are variables *mutable*?

Two Major Concerns

1. Are variables *mutable*?
2. How are variables *scoped*? (Dynamically or lexically?)

Two Major Concerns

1. Are variables *mutable*?
2. How are variables *scoped*? (Dynamically or lexically?)

OCaml variables are immutable, binding defined, and lexically scoped

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Def. *(informal)* A variable is **mutable** if we are allowed to change its value after it has been declared

Scope

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Warning. Scope is one of the most unclear terms in computer science...

Dynamic Scoping

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic Scoping

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

Dynamic Scoping

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

This is a *temporal view*, i.e., what a computation done beforehand which affected the value of a variable

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

» The binding defines it's own scope (let-bindings)

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

- The binding defines it's own scope (let-bindings)
- A block defines the scope of a variable (python functions)

Tradeoffs

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

dynamic

vs.

```
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

lexical

Implementing dynamic scoping is *way* easier...

But **every modern PL** implements lexical scoping

The Environment Model

Motivation

```
let x = v in (... x ... x ... x ...)
```

$$\downarrow_{\beta}$$

```
(... v ... v ... v ...)
```

Motivation

```
let x = v in (... x ... x ... x ...)
```

$$\downarrow_\beta$$

```
(... v ... v ... v ...)
```

The substitution model works, but it's *inefficient*

Motivation

```
let x = v in (... x ... x ... x ...)
```

$$\downarrow_\beta$$

```
(... v ... v ... v ...)
```

The substitution model works, but it's *inefficient*

Each substitution requires us to "crawl" through the *entire remainder of the program*

High Level

$$\mathscr{E} \vdash e \Downarrow v \qquad \textit{or} \qquad \langle \mathscr{E}, e \rangle \Downarrow v$$

High Level

$$\mathscr{E} \vdash e \Downarrow v \quad \textit{or} \quad \langle \mathscr{E}, e \rangle \Downarrow v$$

Idea. We keep track of their values in an *environment*

High Level

$$\mathscr{E} \vdash e \Downarrow v \qquad \textit{or} \qquad \langle \mathscr{E}, e \rangle \Downarrow v$$

Idea. We keep track of their values in an *environment*

We evaluate *relative* to the environment and *lazily* fill in variable values along the way

Lambda Calculus$^+$ (Syntax)

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
         | let <var> = <expr>
           in <expr>
         | <num>

<val>  ::= λ<var>.<expr>
         | <num>
```

This is a grammar for the lambda calculus with let-expressions and numbers

Lambda Calculus^{+} (Semantics)

Important. These rules are incorrect!

Lambda Calculus$^+$ (Semantics)

Important. These rules are incorrect!

"values evaluate to values"

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n}$$

Lambda Calculus$^+$ (Semantics)

Important. These rules are incorrect!

"values evaluate to values"

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n}$$

"variables evaluate to their values in the environment"

$$\frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

Lambda Calculus$^+$ (Semantics)

Important. These rules are incorrect!

"values evaluate to values"

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n}$$

"variables evaluate to their values in the environment"

$$\frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow v_1 \qquad \mathscr{E}_2 = \mathscr{E}_1[x \mapsto v_1] \qquad \mathscr{E}_2 \vdash e_2 \Downarrow v_2}{\mathscr{E}_1 \vdash \text{let } x = e_1 \text{ in } e_2 \Downarrow v_2}$$

"let-expressions store arguments in the environment"

Lambda Calculus$^+$ (Semantics)

Important. These rules are incorrect!

"values evaluate to values"

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n}$$

"variables evaluate to their values in the environment"

$$\frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow v_1 \qquad \mathscr{E}_2 = \mathscr{E}_1[x \mapsto v_1] \qquad \mathscr{E}_2 \vdash e_2 \Downarrow v_2}{\mathscr{E}_1 \vdash \text{let } x = e_1 \text{ in } e_2 \Downarrow v_2}$$

"let-expressions store arguments in the environment"

$$\frac{\mathscr{E} \vdash e_1 \Downarrow \lambda x\,.\,e \qquad \mathscr{E} \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}' = \mathscr{E}[x \mapsto v_2] \qquad \mathscr{E}' \vdash e \Downarrow v}{\mathscr{E} \vdash e_1 e_2 \Downarrow v}$$

"same with applications"

Why are these rules incorrect?

$$\text{let } x = 0 \text{ in}$$
$$\text{let } f = \lambda y \,.\, x \text{ in}$$
$$\text{let } x = 1 \text{ in}$$
$$f\ 0$$

What's the value of this expression?

Closures

Closures

$$(\mathscr{E}, e)$$

Closures

$$(\mathscr{E}, e)$$

Def. *(informal)* A **closure** is an expression plus an environment

Closures

$$(\mathscr{E}, e)$$

Def. *(informal)* A **closure** is an expression plus an environment

It *captures* bindings it needs to evaluate its expression

Closures

$$(\mathscr{E}, e)$$

Def. *(informal)* A **closure** is an expression plus an environment

It *captures* bindings it needs to evaluate its expression

Idea. Functions need to *remember* what the environment looked like in order to behavior correctly (according to lexical scoping)

Lambda Calculus$^+$ (Values)

$$\mathsf{Val} = \mathbb{Z} \cup \mathsf{Cls}$$

Lambda Calculus$^+$ (Values)

$$\mathsf{Val} = \mathbb{Z} \cup \mathsf{Cls}$$

A value is a **number** or a **closure**

Lambda Calculus$^+$ (Values)

$$\mathsf{Val} = \mathbb{Z} \cup \mathsf{Cls}$$

A value is a **number** or a **closure**

Important. *Values no longer correspond to expressions*

Lambda Calculus$^+$ (Correct Semantics)

Lambda Calculus$^+$ (Correct Semantics)

values and variables

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

Lambda Calculus$^+$ (Correct Semantics)

values and variables

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

application

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

Lambda Calculus$^+$ (Correct Semantics)

values and variables

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

application

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

let-expressions

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow v_1 \qquad \mathscr{E}_2 = \mathscr{E}_1[x \mapsto v_1] \qquad \mathscr{E}_2 \vdash e_2 \Downarrow v_2}{\mathscr{E}_1 \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \Downarrow v_2}$$

Recursion

High-Level

```
let f x =
  if x = 0
  then 1
  else f (x - 1)
in f 10
```

High-Level

```
let f x =
  if x = 0
  then 1
  else f (x - 1)
in f 10
```

So far, we've only considered *non-recursive* functions (recursion is difficult...)

High-Level

```
let f x =
  if x = 0
  then 1
  else f (x - 1)
in f 10
```

So far, we've only considered *non-recursive* functions (recursion is difficult...)

In the substitution model, there's no natural way to do it (though we can use fix-point combinators...)

The Problem

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$(\mathscr{E}, \lambda x\,.\,e)$$

The Problem

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$(\mathscr{E}, \lambda x\,.\,e)$$

In order to implement recursion, a closure has to "*know thyself*"

The Problem

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$(\mathscr{E}, \lambda x\,.\,e)$$

In order to implement recursion, a closure has to "*know thyself*"

But we **can't** implement circular structures like this in OCaml

The Problem

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$\{\ldots f \mapsto (\mathscr{E}, \lambda x\,.\,e)\ldots\}$$

$$(\mathscr{E}, \lambda x\,.\,e)$$

In order to implement recursion, a closure has to "*know thyself*"

But we **can't** implement circular structures like this in OCaml

We need a way essentially to "simulate" pointers

Named Closures

$$(\mathscr{E}, \mathsf{name} \mapsto \lambda x\,.\,e)$$

We need to be able to *name* closures

Idea. Named closures will put themselves into their environment *when they're called*

Lambda Calculus^{++} (Syntax, Again)

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
         | let <var> = <expr>
           in <expr>
         | let rec <var> <var> = <expr>
           in <expr>
         | <num>
```

The same grammar as before, but with recursive let-statements

Important. A recursive let **must** take an argument

Lambda Calculus^{++} (Semantics)

Lambda Calculus^{++} (Semantics)

values and variables

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

Lambda Calculus^{++} (Semantics)

values and variables

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

application (unnamed closure)

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \cdot \mapsto \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

Lambda Calculus^{++} (Semantics)

values and variables

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

application (unnamed closure)

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \cdot \mapsto \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

application (named closure)

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, f \mapsto \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[f \mapsto (\mathscr{E}_2, f \mapsto \lambda x\,.\,e)][x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

Lambda Calculus^{++} (Semantics)

values and variables

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

application (unnamed closure)

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \cdot \mapsto \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

application (named closure)

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, f \mapsto \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[f \mapsto (\mathscr{E}_2, f \mapsto \lambda x\,.\,e)][x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

let expressions

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow v_1 \qquad \mathscr{E}_2 = \mathscr{E}_1[x \mapsto v_1] \qquad \mathscr{E}_2 \vdash e_2 \Downarrow v_2}{\mathscr{E}_1 \vdash \text{let } x = e_1 \text{ in } e_2 \Downarrow v_2}$$

$$\frac{\mathscr{E}_2 = \mathscr{E}_1[f \mapsto (\mathscr{E}_1, f \mapsto \lambda x\,.\,e_1)] \qquad \mathscr{E}_2 \vdash e_2 \Downarrow v_2}{\mathscr{E}_1 \vdash \text{let rec } f\ x = e_1 \text{ in } e_2 \Downarrow v_2}$$

workshop